C++ 线程管控

Posted on 2023-08-13 12:16:01 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2Tuk4RfvfBC788LlqnQrWiPiEGW)

简介

本节介绍C++线程管控，包括移交线程的归属权，线程并发数量控制以及获取线程id等基本操作。

线程归属权

我们之前介绍了线程可以通过detach在后台运行或者让开辟这个线程的父线程等待该线程完成。
但每个线程都应该有其归属权，也就是归属给某个变量管理。比如

```
1. void some_function() {
2.
3. }
4.
5. std::thread t1(some_function);
```

t1是一个线程变量，管理一个线程，该线程执行 `some_function()`
对于 `std::thread` C++ 不允许其执行拷贝构造和拷贝赋值, 所以只能通过移动和局部变量返回的方式将线程变量管理的线程转移给其他变量管理。
C++ 中类似的类型还有 `std::mutex` ， `std::ifstream` ， `std::unique_ptr` 。
比如下面，我们说明了线程归属权的转移方式

```
1. void some_function() {
2.     while (true) {
3.         std::this_thread::sleep_for(std::chrono::seconds(1));
4.     }
5. }
6.
7. void some_other_function() {
8.     while (true) {
9.         std::this_thread::sleep_for(std::chrono::seconds(1));
10.    }
11. }
12.
13. //t1 绑定some_function
14. std::thread t1(some_function);
15. //2 转移t1管理的线程给t2，转移后t1无效
16. std::thread t2 =  std::move(t1);
17. //3 t1 可继续绑定其他线程,执行some_other_function
18. t1 = std::thread(some_other_function);
19. //4  创建一个线程变量t3
20. std::thread t3;
21. //5  转移t2管理的线程给t3
22. t3 = std::move(t2);
23. //6  转移t3管理的线程给t1
24. t1 = std::move(t3);
25. std::this_thread::sleep_for(std::chrono::seconds(2000));
```

上面的代码会引发崩溃，是因为步骤6造成的崩溃。
让主函数睡眠2000秒，是为了告诉规避主函数退出引发崩溃的问题，因为我们在之前给大家演示过，如果线程不detach或者join，主线程退出时会引发崩溃，而我们这些线程没有join和detach，为了给大家演示是因为步骤6引发的崩溃，所以让主线程睡眠2000秒暂时不退出，但是程序仍然会崩溃，说明是步骤6导致的崩溃。

上面代码将t2管理的线程交给t3
之后将t3管理的线程交给t1，此时t1管理线程运行着 `some_function`
步骤6导致崩溃的原因就是将t3管理的线程交给t1，而此时t1正在管理线程运行 `some_other_function` 。
所以我们可以得出一个结论，就是不要将一个线程的管理权交给一个已经绑定线程的变量，否则会触发线程的terminate函数引发崩溃。

和 `std::unique_ptr` 一样，我们可以在函数内部返回一个局部的 `std::thread` 变量。如下:

```
std::thread  f() {
    return std::thread(some_function);
}

void param_function(int a) {
    while (true) {
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }
}

std::thread g() {
    std::thread t(param_function, 43);
}
```

因为C++ 在返回局部变量时，会优先寻找这个类的拷贝构造函数，如果没有就会使用这个类的移动构造函数。

joining_thread

曾经有一份C++17标准的备选提案，主张引入新的类joining_thread，它与std::thread类似，但只要其执行析构函数，线程即能自动汇合，这点与scoped_thread非常像。可惜C++标准委员会未能达成共识，结果C++17标准没有引入这个类，后来它改名为std::jthread，依然进入了C++20标准的议程（现已被正式纳入C++20标准）。除去这些，实际上joining_thread类的代码相对容易编写

```
1. class joining_thread {
2.     std::thread  _t;
3. public:
4.     joining_thread() noexcept = default;
5.     template<typename Callable, typename ...  Args>
6.     explicit  joining_thread(Callable&& func, Args&& ...args):
7.         t(std::forward<Callable>(func),  std::forward<Args>(args)...){}
8.     explicit joining_thread(std::thread  t) noexcept: _t(std::move(t)){}
9.     joining_thread(joining_thread&& other) noexcept: _t(std::move(other._t)){}
10.    joining_thread& operator=(joining_thread&& other) noexcept
11.    {
12.        //如果当前线程可汇合，则汇合等待线程完成再赋值
13.        if (joinable()) {
14.            join();
15.        }
16.        _t = std::move(other._t);
17.        return *this;
18.    }
19.
20.    joining_thread& operator=(joining_thread other) noexcept
21.    {
22.        //如果当前线程可汇合，则汇合等待线程完成再赋值
23.        if (joinable()) {
24.            join();
25.        }
26.        _t = std::move(other._t);
27.        return *this;
28.    }
29.
30.
31.    ~joining_thread() noexcept {
32.        if (joinable()) {
33.            join();
34.        }
35.    }
36.
37.    void swap(joining_thread& other) noexcept {
38.        _t.swap(other._t);
39.    }
40.
41.    std::thread::id   get_id() const noexcept {
42.        return _t.get_id();
```

```
    }

    bool joinable() const noexcept {
        return _t.joinable();
    }

    void join() {
        _t.join();
    }

    void detach() {
        _t.detach();
    }

    std::thread& as_thread() noexcept {
        return _t;
    }

    const std::thread& as_thread() const noexcept {
        return _t;
    }
};
```

使用起来比较简单，我们直接构造一个joining_thread对象即可。

```
1. void use_jointhread() {
2.     //1 根据线程构造函数构造joiningthread
3.     joining_thread j1([](int maxindex) {
4.         for (int i = 0; i < maxindex; i++) {
5.             std::cout << "in thread id " << std::this_thread::get_id()
6.                 << " cur index is " << i << std::endl;
7.             std::this_thread::sleep_for(std::chrono::seconds(1));
8.         }
9.         }, 10);
10.
11.    //2 根据thread构造joiningthread
12.    joining_thread j2(std::thread([](int maxindex) {
13.        for (int i = 0; i < maxindex; i++) {
14.            std::cout << "in thread id " << std::this_thread::get_id()
15.                << " cur index is " << i << std::endl;
16.            std::this_thread::sleep_for(std::chrono::seconds(1));
17.        }
18.        }, 10));
19.
20.    //3 根据thread构造j3
21.    joining_thread j3(std::thread([](int maxindex) {
22.        for (int i = 0; i < maxindex; i++) {
23.            std::cout << "in thread id " << std::this_thread::get_id()
24.                << " cur index is " << i << std::endl;
25.            std::this_thread::sleep_for(std::chrono::seconds(1));
26.        }
27.        }, 10));
28.
29.
30.    //4 把j3赋值给j1，joining_thread内部会等待j1汇合结束后
31.    //再将j3赋值给j1
32.    j1 = std::move(j3);
33. ervice
34. }
```

容器存储

容器存储线程时，比如 `vector` ,如果用 `push_back` 操作势必会调用 `std::thread` ，这样会引发编译错误，因为 `std::thread` 没有拷贝构造函数。我们在之前网络编程实现IOServicePool或者IOThreadPool时初始化了多个线程存储在vector中, 采用的时emplace方式，可以直接根

据线程构造函数需要的参数构造，这样就避免了调用thread的拷贝构造函数。 类似于这种

```
1. void use_vector() {
2.     std::vector<std::thread> threads;
3.     for (unsigned i = 0; i < 10; ++i) {
4.         threads.emplace_back(param_function, i);
5.     }
6.
7.     for (auto& entry : threads) {
8.         entry.join();
9.     }
10. }
```

选择运行数量

借用C++标准库的 `std::thread::hardware_concurrency()` 函数，它的返回值是一个指标，表示程序在各次运行中可真正并发的线程数量.
我们可以模拟实现一个并行计算的功能，计算容器内所有元素的和

```
1. template<typename Iterator, typename T>
2. T parallel_accumulate(Iterator first, Iterator last, T init)
3. {
4.     unsigned long const length = std::distance(first, last);
5.     if (!length)
6.         return init;    //<--- - ①
7.         unsigned long const min_per_thread = 25;
8.     unsigned long const max_threads =
9.         (length + min_per_thread - 1) / min_per_thread;    //<--- - ②
10.        unsigned long const hardware_threads =
11.        std::thread::hardware_concurrency();
12.    unsigned long const num_threads =
13.        std::min(hardware_threads != 0 ? hardware_threads : 2, max_threads);    //<--- - ③
14.        unsigned long const block_size = length / num_threads;    //<--- - ④
15.        std::vector<T> results(num_threads);
16.    std::vector<std::thread>  threads(num_threads - 1);   // <--- - ⑤
17.        Iterator block_start = first;
18.    for (unsigned long i = 0; i < (num_threads - 1); ++i)
19.    {
20.        Iterator block_end = block_start;
21.        std::advance(block_end, block_size);    //<--- - ⑥
22.            threads[i] = std::thread(//<--- - ⑦
23.                accumulate_block<Iterator, T>(),
24.                block_start, block_end, std::ref(results[i]));
25.        block_start = block_end;    //<--- - ⑧
26.    }
27.    accumulate_block<Iterator, T>()(
28.        block_start, last, results[num_threads - 1]);    //<--- - ⑨
29.
30.        for (auto& entry : threads)
31.            entry.join();    //<--- - ⑩
32.            return std::accumulate(results.begin(), results.end(), init);    //<--- - ⑪
33. }
34.
35. void use_parallel_acc() {
36.    std::vector <int> vec(10000);
37.    for (int i = 0; i < 10000; i++) {
38.        vec.push_back(i);
39.    }
40.    int sum = 0;
41.    sum = parallel_accumulate<std::vector<int>::iterator, int>(vec.begin(),
42.        vec.end(), sum);
```

```
43.
44.     std::cout << "sum is " << sum << std::endl;
45. }
```

上面的代码1处判断要计算的容器内元素为0个则返回。

2处计算最大开辟的线程数，我们预估每个线程计算25个数据长度。

但是我们可以通过 `std::thread::hardware_concurrency` 返回cpu的核数，我们期待的是开辟的线

程数小于等于cpu核数，这样才不会造成线程过多时间片切换开销。

所以3处计算了适合开辟线程数的最小值。

4处计算了步长，根据步长移动迭代器然后开辟线程计算。

5处初始化了线程数-1个大小的vector，因为主线程也参与计算，所以这里-1.

6处移动步长，7处开辟线程，8处更新起始位置。

9处为主线程计算。

10 处让所有线程join

11 处最后将所有计算结果再次调用std的accumulate算出结果。

识别线程

所谓识别线程就是获取线程id，可以根据线程id是否相同判断是否同一个线程。
比如我们启动了一个线程,我们可以通过线程变量的 `get_id()` 获取线程id

```
1. std::thread t([](){
2.     std::cout << "thread start" << std::endl;
3. });
4.
5. t.get_id();
```

但是如果我们想在线程的运行函数中区分线程，或者判断哪些是主线程或者子线程，可以通过这总方式

```
1. std::thread t([](){
2.     std::cout << "in thread id " <<
3.     std::this_thread::get_id() << std::endl;
4.     std::cout << "thread start" << std::endl;
5. });
```

总结

本文介绍了线程管控相关的知识，包括线程的join,detach，以及识别线程，归属权转移，如何管理等等。

本文介绍了 `std::thread` 的基本操作，具体视频可以去B站看看我的C++视频讲解

https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101&ctype=0 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101&ctype=0)

代码链接

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(4636) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(559) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论



搁浅

2024-01-09 10:10:40

函数名错误：

上面代码将t2管理的线程交给t3

之后将t3管理的线程交给t1，此时t1管理线程运行着 `some_function`
步骤6导致崩溃的原因就是将t3管理的线程交给t1，而此时t1正在管理线程运行 `some_function`

修正：

t1管理运行的是some_other_function，t3才是运行管理的some_function

♡ 喜欢(1) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看 ≫

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态
gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数
李程浩:1

C++ 线程管控
secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础
secondtonone1:谢谢，笔误已修复

再谈单例模式
secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)